Lecture 2. Introduction to Racket. Expressions, Pairs, and Lists. The Substitution Model

Programming Paradigms, Fall 2026 — 2 Sep 2026

Nikolai Kudasov

Lab of Programming Languages and Compilers

Innopolis University



Outline

1. From λ-calculus to Racket
2. Racket Basics: Definitions and Expressions
3. The Substitution Model
4. Lists
5. Recursive Functions over Lists

From λ-calculus to Racket

About Racket

1. Racket is a dynamically typed functional programming language.
2. Racket is a dialect of LISP (more specifically, of Scheme).
3. Racket has a dedicated IDE (called DrRacket).
4. We will use Racket to learn about
   - programming with (pure) functions
   - managing local state with an accumulator parameter
   - working with higher-order functions
   - iterators and list comprehensions
   - thinking about programs via the substitution model

The λ-calculus, in Racket Syntax

Racket is the λ-calculus with constants, written with more parentheses.

	λ-calculus	Racket
abstraction	$\lambda x.\, t$	`(lambda (x) t)`
application	$t\ u$	`(t u)`

The λ-calculus, in Racket Syntax

Racket is the λ-calculus with constants, written with more parentheses.

	λ-calculus	Racket
abstraction	$\lambda x.\, t$	`(lambda (x) t)`
application	$t\ u$	`(t u)`
two arguments	$\lambda x.\, \lambda y.\, t$ and $f\ a\ b$	`(lambda (x y) t)` and `(f a b)`

The λ-calculus, in Racket Syntax

Racket is the λ-calculus with constants, written with more parentheses.

	λ-calculus	Racket
abstraction	$\lambda x.\, t$	`(lambda (x) t)`
application	$t\ u$	`(t u)`
two arguments	$\lambda x.\, \lambda y.\, t$ and $f\ a\ b$	`(lambda (x y) t)` and `(f a b)`
a name for a term	$\mathsf{id} = \lambda x.\, x$	`(define id (lambda (x) x))`

The λ-calculus, in Racket Syntax



Racket is the λ-calculus with constants, written with more parentheses.

	λ-calculus	Racket
abstraction	$\lambda x.\, t$	`(lambda (x) t)`
application	$t\ u$	`(t u)`
two arguments	$\lambda x.\, \lambda y.\, t$ and $f\ a\ b$	`(lambda (x y) t)` and `(f a b)`
a name for a term	$\mathrm{id} = \lambda x.\, x$	`(define id (lambda (x) x))`
constants	1, $+$ (an extension)	`1`, `+`, `"text"` (built in)

The λ-calculus, in Racket Syntax



Racket is the λ-calculus with constants, written with more parentheses.

	λ-calculus	Racket
abstraction	$\lambda x.\, t$	`(lambda (x) t)`
application	$t\ u$	`(t u)`
two arguments	$\lambda x.\, \lambda y.\, t$ and $f\ a\ b$	`(lambda (x y) t)` and `(f a b)`
a name for a term	$\mathrm{id} = \lambda x.\, x$	`(define id (lambda (x) x))`
constants	1, $+$ (an extension)	`1`, `+`, `"text"` (built in)
β-reduction	$(\lambda x.\, t)\ u \longrightarrow [x \mapsto u]\, t$	the **substitution model**, today

The λ-calculus, in Racket Syntax



Racket is the λ-calculus with constants, written with more parentheses.

	λ-calculus	Racket
abstraction	$\lambda x.\, t$	`(lambda (x) t)`
application	$t\ u$	`(t u)`
two arguments	$\lambda x.\, \lambda y.\, t$ and $f\ a\ b$	`(lambda (x y) t)` and `(f a b)`
a name for a term	$\mathrm{id} = \lambda x.\, x$	`(define id (lambda (x) x))`
constants	1, $+$ (an extension)	`1`, `+`, `"text"` (built in)
β-reduction	$(\lambda x.\, t)\ u \longrightarrow [x \mapsto u]\, t$	the **substitution model**, today

Racket reduces the arguments first, and never under a **`lambda`**.
That is **call by value**; SICP calls it *applicative order*.

Reading Racket Out Loud

```
(+ (* 2 3) (- 10 4))
```

Reading Racket Out Loud

```
(+ (* 2 3) (- 10 4))
```

Add the product of 2 and 3 to the difference of 10 and 4. `; 12`

Reading Racket Out Loud

```
(+ (* 2 3) (- 10 4))
```

Add the product of 2 and 3 to the difference of 10 and 4. `; 12`

- Application is written in parentheses, **function first**: `(f a b)` is *f a b*.

Reading Racket Out Loud

```
(+ (* 2 3) (- 10 4))
```

Add the product of 2 and 3 to the difference of 10 and 4. `; 12`

- Application is written in parentheses, **function first**: `(f a b)` is *f a b*.
- Not optional: `(f)` is a call with no arguments, `f` alone is the function.

Reading Racket Out Loud

```
(+ (* 2 3) (- 10 4))
```

Add the product of 2 and 3 to the difference of 10 and 4. `; 12`

- Application is written in parentheses, **function first**: `(f a b)` is *f a b*.
- Not optional: `(f)` is a call with no arguments, `f` alone is the function.
- No infix operators, so there is no precedence to remember.

```
(+ (* 2 3) (- 10 4))
```

Add the product of 2 and 3 to the difference of 10 and 4. `; 12`

- Application is written in parentheses, **function first**: `(f a b)` is *f a b*.
- Not optional: `(f)` is a call with no arguments, `f` alone is the function.
- No infix operators, so there is no precedence to remember.

Question

Read `((lambda (x) (* x x)) 3)` out loud, then evaluate it.

Racket Basics: Definitions and Expressions

Racket Definitions

Read “Quick: An Introduction to Racket with Pictures”[1].

```
(define a-disk
  (colorize (disk 90) "blue"))
```

[1] See https://docs.racket-lang.org/quick/index.html.

Racket Function Definitions

Read “Quick: An Introduction to Racket with Pictures”[2].

```
(define (square width color)
  (colorize
    (filled-rectangle width width)
    color))
```

[2] See https://docs.racket-lang.org/quick/index.html.

Racket Function Definitions

Read “Quick: An Introduction to Racket with Pictures”[2].

```
(define (square width color)
  (colorize
    (filled-rectangle width width)
    color))
```

This is shorthand for a **`define`** of a **`lambda`**:

```
(define square
  (lambda (width color)
    (colorize (filled-rectangle width width) color)))
```

[2] See https://docs.racket-lang.org/quick/index.html.

Racket Function Calls

Read “Quick: An Introduction to Racket with Pictures”[3].

```
(square 100 "blue")

(square (+ 50 50) "blue")
```

[3] See https://docs.racket-lang.org/quick/index.html.

Racket Conditionals

cond tries its clauses in order, and evaluates the body of the first one whose test holds.

else catches everything left.

```
(define (quantity-to-text word n)
  (cond
    [(= n 1)    (string-append "one " word)]
    [(<= 2 n 3) (string-append "a couple of " word "s")]
    [else       (string-append "many " word "s")]))
```

Racket Conditionals

`cond` tries its clauses in order, and evaluates the body of the first one whose test holds.
`else` catches everything left.

```
(define (quantity-to-text word n)
  (cond
    [(= n 1)    (string-append "one " word)]
    [(<= 2 n 3) (string-append "a couple of " word "s")]
    [else       (string-append "many " word "s")]))
```

A test is any expression that yields `#t` or `#f`.
Comparisons take any number of arguments: `(<= 2 n 3)` is $2 \leq n \leq 3$.

Racket Conditionals (Example Calls)

```
(quantity-to-text "apple"  1)
; "one apple"

(quantity-to-text "orange" 2)
; "a couple of oranges"

(quantity-to-text "banana" 4)
; "many bananas"
```

Anonymous Functions

Read “Quick: An Introduction to Racket with Pictures”[4].

```
(define (twice f x)
  (f (f x)))

(twice (lambda (x) (* x x)) 2) ; 16
```

[4] See https://docs.racket-lang.org/quick/index.html.

Local Names: `let`

`let` names the values of some expressions, for its body only[5]:

```
(define (hypotenuse a b)
  (let ([a2 (* a a)]
        [b2 (* b b)])
    (sqrt (+ a2 b2))))

(hypotenuse 3 4) ; 5
```

[5] See Racket Essentials §2.2.8.

Local Names: `let`



`let` names the values of some expressions, for its body only[5]:

```
(define (hypotenuse a b)
  (let ([a2 (* a a)]
        [b2 (* b b)])
    (sqrt (+ a2 b2))))

(hypotenuse 3 4) ; 5
```

A `let` is a `lambda` applied on the spot:
`(let ([x e]) body)` is `((lambda (x) body) e)`.

[5] See Racket Essentials §2.2.8.

Local Names: `let`



`let` names the values of some expressions, for its body only[5]:

```
(define (hypotenuse a b)
  (let ([a2 (* a a)]
        [b2 (* b b)])
    (sqrt (+ a2 b2))))

(hypotenuse 3 4) ; 5
```

A **`let`** is a **`lambda`** applied on the spot:

`(let ([x e]) body)` is `((lambda (x) body) e)`.

So the substitution model needs no new rule for it: substitute `e` for `x` in `body`.

[5] See Racket Essentials §2.2.8.

Where a Name Gets Its Meaning

```
(define scale 10)
(define (stretch n) (* n scale))

(let ([scale 2]) (stretch 5))
```

Where a Name Gets Its Meaning

```
(define scale 10)
(define (stretch n) (* n scale))

(let ([scale 2]) (stretch 5))
```

50 or 10?

Where a Name Gets Its Meaning

```
(define scale 10)
(define (stretch n) (* n scale))

(let ([scale 2]) (stretch 5))
```

50 or **10**?

```
((lambda (scale) (stretch 5)) 2)
```

Where a Name Gets Its Meaning

```
(define scale 10)
(define (stretch n) (* n scale))

(let ([scale 2]) (stretch 5))
```

50 or 10?

```
((lambda (scale) (stretch 5)) 2)
= (stretch 5)
```

no free `scale` here

Where a Name Gets Its Meaning

```
(define scale 10)
(define (stretch n) (* n scale))

(let ([scale 2]) (stretch 5))
```

50 or 10?

```
((lambda (scale) (stretch 5)) 2)
= (stretch 5)        no free scale here
= (* 5 scale)        the top-level scale
```

Where a Name Gets Its Meaning



```
(define scale 10)
(define (stretch n) (* n scale))

(let ([scale 2]) (stretch 5))
```

50 or 10?

```
((lambda (scale) (stretch 5)) 2)
= (stretch 5)        no free scale here
= (* 5 scale)        the top-level scale
= (* 5 10) = 50
```

Where a Name Gets Its Meaning

```
(define scale 10)
(define (stretch n) (* n scale))

(let ([scale 2]) (stretch 5))
```

`50` or `10`?

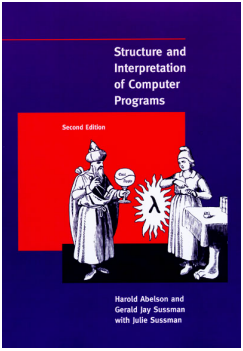
```
((lambda (scale) (stretch 5)) 2)
= (stretch 5)        no free scale here
= (* 5 scale)        the top-level scale
= (* 5 10) = 50
```

Lexical scope

A name means what it meant where the function was **written**.
Inlining `stretch` under the **`let`** gives `(* 5 2)`: capture, as in lecture 1.

The Substitution Model

The Wizard Book



Harold Abelson and Gerald Jay Sussman (1996). ***Structure and interpretation of computer programs.*** The MIT Press

The Substitution Model for Procedure Application

To apply a compound procedure to arguments, evaluate the body of the procedure with each formal parameter replaced by the corresponding argument.

— SICP (Abelson and Sussman 1996, §1.1.5)

The Substitution Model for Procedure Application

> *To apply a compound procedure to arguments, evaluate the body of the procedure with each formal parameter replaced by the corresponding argument.*
>
> — SICP (Abelson and Sussman 1996, §1.1.5)

1. The substitution model helps us think and **reason about programs** (semantically), **not** to understand how the interpreter works!

The Substitution Model

The Substitution Model for Procedure Application

> *To apply a compound procedure to arguments, evaluate the body of the procedure with each formal parameter replaced by the corresponding argument.*
>
> — SICP (Abelson and Sussman 1996, §1.1.5)

1. The substitution model helps us think and **reason about programs** (semantically), **not** to understand how the interpreter works!
2. We will crucially depend on *referential transparency*.
   Without it the substitution model no longer works in general, see “The Cost of Introducing Assignment” (Abelson and Sussman 1996, §3.1.3).

The Substitution Model (Example 1)

```
1  (define (square x) (* x x))
2  (define (sum-of-squares x y)
3    (+ (square x) (square y)))
4  (define (f z)
5    (sum-of-squares (+ z 2) (* z 3)))

   (f 5)
```

The Substitution Model (Example 1)

```
1  (define (square x) (* x x))
2  (define (sum-of-squares x y)
3    (+ (square x) (square y)))
4  (define (f z)
5    (sum-of-squares (+ z 2) (* z 3)))
```

```
(f 5)
= (sum-of-squares (+ 5 2) (* 5 3))
```

The Substitution Model (Example 1)

```
1  (define (square x) (* x x))
2  (define (sum-of-squares x y)
3    (+ (square x) (square y)))
4  (define (f z)
5    (sum-of-squares (+ z 2) (* z 3)))
```

```
(f 5)
= (sum-of-squares (+ 5 2) (* 5 3))
= (sum-of-squares 7 15)
```

The Substitution Model (Example 1)

```
1  (define (square x) (* x x))
2  (define (sum-of-squares x y)
3    (+ (square x) (square y)))
4  (define (f z)
5    (sum-of-squares (+ z 2) (* z 3)))
```

```
(f 5)
= (sum-of-squares (+ 5 2) (* 5 3))
= (sum-of-squares 7 15)
= (+ (square 7) (square 15))
```

The Substitution Model (Example 1)

```
1  (define (square x) (* x x))
2  (define (sum-of-squares x y)
3    (+ (square x) (square y)))
4  (define (f z)
5    (sum-of-squares (+ z 2) (* z 3)))
```

```
(f 5)
= (sum-of-squares (+ 5 2) (* 5 3))
= (sum-of-squares 7 15)
= (+ (square 7) (square 15))
= (+ (* 7 7) (* 15 15))
```

The Substitution Model (Example 1)

```
1  (define (square x) (* x x))
2  (define (sum-of-squares x y)
3    (+ (square x) (square y)))
4  (define (f z)
5    (sum-of-squares (+ z 2) (* z 3)))
```

```
(f 5)
= (sum-of-squares (+ 5 2) (* 5 3))
= (sum-of-squares 7 15)
= (+ (square 7) (square 15))
= (+ (* 7 7) (* 15 15))
= (+ 49 225)
```

The Substitution Model (Example 1)

```
1  (define (square x) (* x x))
2  (define (sum-of-squares x y)
3    (+ (square x) (square y)))
4  (define (f z)
5    (sum-of-squares (+ z 2) (* z 3)))
```

```
(f 5)
= (sum-of-squares (+ 5 2) (* 5 3))
= (sum-of-squares 7 15)
= (+ (square 7) (square 15))
= (+ (* 7 7) (* 15 15))
= (+ 49 225)
= 274
```

The Substitution Model (Example 2)

```
1  (define (twice f x)
2    (f (f x)))
```

```
(twice (lambda (x) (* x x)) 2)
```

The Substitution Model (Example 2)

```
1 (define (twice f x)
2   (f (f x)))
```

```
(twice (lambda (x) (* x x)) 2)
= ((lambda (x) (* x x)) ((lambda (x) (* x x)) 2))
```

The Substitution Model (Example 2)

```
1 (define (twice f x)
2   (f (f x)))
```

```
(twice (lambda (x) (* x x)) 2)
= ((lambda (x) (* x x)) ((lambda (x) (* x x)) 2))
= ((lambda (x) (* x x)) (* 2 2))
```

The Substitution Model (Example 2)

```
1 (define (twice f x)
2   (f (f x)))
```

```
(twice (lambda (x) (* x x)) 2)
= ((lambda (x) (* x x)) ((lambda (x) (* x x)) 2))
= ((lambda (x) (* x x)) (* 2 2))
= ((lambda (x) (* x x)) 4)
```

The Substitution Model (Example 2)

```
1 (define (twice f x)
2   (f (f x)))
```

```
(twice (lambda (x) (* x x)) 2)
= ((lambda (x) (* x x)) ((lambda (x) (* x x)) 2))
= ((lambda (x) (* x x)) (* 2 2))
= ((lambda (x) (* x x)) 4)
= (* 4 4)
```

The Substitution Model (Example 2)

```
1  (define (twice f x)
2    (f (f x)))
```

```
(twice (lambda (x) (* x x)) 2)
= ((lambda (x) (* x x)) ((lambda (x) (* x x)) 2))
= ((lambda (x) (* x x)) (* 2 2))
= ((lambda (x) (* x x)) 4)
= (* 4 4)
= 16
```

The Substitution Model (Example 3a)

```
1 (define (make-twice f)
2   (lambda (x) (f (f x))))

((make-twice g) z)
```

The Substitution Model (Example 3a)

```
1 (define (make-twice f)
2   (lambda (x) (f (f x))))
```

```
((make-twice g) z)
= ((lambda (x) (g (g x))) z)
```

The Substitution Model (Example 3a)

```
1 (define (make-twice f)
2   (lambda (x) (f (f x))))
```

```
((make-twice g) z)
= ((lambda (x) (g (g x))) z)
= (g (g z))
```

The Substitution Model (Example 3a)

```
1 (define (make-twice f)
2   (lambda (x) (f (f x))))
```

```
((make-twice g) z)
= ((lambda (x) (g (g x))) z)
= (g (g z))
```

Note that this works for any (pure) `g` and `z`.

The Substitution Model (Example 3b)

```
1 (define (make-twice f)
2   (lambda (x) (f (f x))))
```

```
(((make-twice make-twice) f) x)
```

The Substitution Model (Example 3b)

```
1 (define (make-twice f)
2   (lambda (x) (f (f x))))
```

```
(((make-twice make-twice) f) x)
= ((make-twice (make-twice f)) x)
```

The Substitution Model (Example 3b)

```
1 (define (make-twice f)
2   (lambda (x) (f (f x))))
```

```
(((make-twice make-twice) f) x)
= ((make-twice (make-twice f)) x)
= ((make-twice f) ((make-twice f) x))
```

The Substitution Model (Example 3b)

```
1 (define (make-twice f)
2   (lambda (x) (f (f x))))
```

```
(((make-twice make-twice) f) x)
= ((make-twice (make-twice f)) x)
= ((make-twice f) ((make-twice f) x))
= ((make-twice f) (f (f x)))
```

The Substitution Model (Example 3b)

```
1 (define (make-twice f)
2   (lambda (x) (f (f x))))
```

```
(((make-twice make-twice) f) x)
= ((make-twice (make-twice f)) x)
= ((make-twice f) ((make-twice f) x))
= ((make-twice f) (f (f x)))
= (f (f (f (f x))))
```

Lists

Racket Lists

Lists in Racket can be constructed using the function `list`:

```
(list 1 2 3 4 5)

(list "apples" "bananas" "oranges")

(list 1 (list 2 3) (list (list)))
```

Lists in Racket are **heterogeneous** (may contain elements of different types).

Standard Functions on Lists



```
(define example
  (list "apples" "bananas" "oranges"))

(length example)      ; 3
(list-ref example 1)  ; "bananas"
(reverse example)     ; '("oranges" "bananas" "apples")
(append example example)
; '("apples" "bananas" "oranges" "apples" "bananas" "oranges")
```

Deconstructing Lists

- `first` (or `car`) takes the first element of the list.
- `rest` (or `cdr`) takes the remaining part of the list.

```
(define example
  (list "apples" "bananas" "oranges"))

(first example) ; "apples"
(rest example)  ; '("bananas" "oranges")

(car example) ; "apples"
(cdr example) ; '("bananas" "oranges")
```

Constructing Lists

`cons` constructs a new list given its first element (head) and its tail.

```
empty                ; '()

(cons 1 (list 2 3)) ; '(1 2 3)

(cons 1 (cons 2 (cons 3 empty)))
```

Checking Structure of Lists

- `empty?` checks if a list is empty.
- `cons?` checks that a list is non-empty[6].

```
(empty? (list 2 3))                ; #f

(empty? (rest (rest (list 2 3)))) ; #t

(cons? (list 2 3))                 ; #t
```

[6] Actually, this is a more general predicate, as we will see in a minute.

Pairs in Racket

`cons` takes two values and constructs a pair:

```
(cons 1 2)           ; '(1 . 2)
(cons "hi" "world") ; '("hi" . "world")
```

`pair?` is the same as `cons?`:

```
(pair? (cons 1 2))  ; #t
```

A list[7] is either empty or a pair of a value and a list:

```
(cons 1 empty)       ; '(1)      same as '(1 . ())
(cons 1 (list 2 3)) ; '(1 2 3)  same as '(1 . (2 . (3)))
```

[7] See Racket Essentials §2.4.

List Components

Helper functions exist[8] to extract a certain element of a list.
There are also short-named combinations of `car` and `cdr`.

```
(first  (list 1 2 3)) ; 1
(second (list 1 2 3)) ; 2
(third  (list 1 2 3)) ; 3

(rest (list 1 2 3)) ; '(2 3)
```

```
(car    (list 1 2 3))
(cadr   (list 1 2 3))
(caddr  (list 1 2 3))

(rest (list 1 2 3))
```

[8] See Racket Essentials §2.4.

Quotation

Note that Racket outputs many values with a single quote:

```
(list (list 1) (list 2 3) (list 4)) ; '((1) (2 3) (4))
```

This is shorthand for application of **`quote`**[9]:

```
(quote ((1) (2 3) (4)))             ; '((1) (2 3) (4))
'((1) (2 3) (4))                    ; '((1) (2 3) (4))
```

The argument of **`quote`** is **not** evaluated and is effectively treated as data (not code).

Note that **`quote`** is **not** a function.
It is a *special form* (a macro), and has to be explicitly applied.

[9] See Racket Essentials §2.4.

5 min break



Recursive Functions over Lists

Sum of a List (Naïve Recursive)

```
1  (define (my-sum lst)
2    (cond
3      [(empty? lst) 0]
4      [else (+ (first lst)
5               (my-sum (rest lst)))]))

   (my-sum (list 1 2 3))
```

Sum of a List (Naïve Recursive)

```
1  (define (my-sum lst)
2    (cond
3      [(empty? lst) 0]
4      [else (+ (first lst)
5               (my-sum (rest lst)))]))
```

```
(my-sum (list 1 2 3))
= (+ 1 (my-sum (list 2 3)))
```

Sum of a List (Naïve Recursive)

```
1  (define (my-sum lst)
2    (cond
3      [(empty? lst) 0]
4      [else (+ (first lst)
5               (my-sum (rest lst)))]))
```

```
(my-sum (list 1 2 3))
= (+ 1 (my-sum (list 2 3)))
= (+ 1 (+ 2 (my-sum (list 3))))
```

Sum of a List (Naïve Recursive)

```
1  (define (my-sum lst)
2    (cond
3      [(empty? lst) 0]
4      [else (+ (first lst)
5               (my-sum (rest lst)))]))
```

```
(my-sum (list 1 2 3))
= (+ 1 (my-sum (list 2 3)))
= (+ 1 (+ 2 (my-sum (list 3))))
= (+ 1 (+ 2 (+ 3 (my-sum empty))))
```

Sum of a List (Naïve Recursive)

```
1  (define (my-sum lst)
2    (cond
3      [(empty? lst) 0]
4      [else (+ (first lst)
5               (my-sum (rest lst)))]))
```

```
(my-sum (list 1 2 3))
= (+ 1 (my-sum (list 2 3)))
= (+ 1 (+ 2 (my-sum (list 3))))
= (+ 1 (+ 2 (+ 3 (my-sum empty))))
= (+ 1 (+ 2 (+ 3 0)))
```

Sum of a List (Naïve Recursive)

```
1  (define (my-sum lst)
2    (cond
3      [(empty? lst) 0]
4      [else (+ (first lst)
5               (my-sum (rest lst)))]))
```

```
(my-sum (list 1 2 3))
= (+ 1 (my-sum (list 2 3)))
= (+ 1 (+ 2 (my-sum (list 3))))
= (+ 1 (+ 2 (+ 3 (my-sum empty))))
= (+ 1 (+ 2 (+ 3 0)))
= (+ 1 (+ 2 3)) = (+ 1 5)
```

Sum of a List (Naïve Recursive)

```
1  (define (my-sum lst)
2    (cond
3      [(empty? lst) 0]
4      [else (+ (first lst)
5               (my-sum (rest lst)))]))
```

```
(my-sum (list 1 2 3))
= (+ 1 (my-sum (list 2 3)))
= (+ 1 (+ 2 (my-sum (list 3))))
= (+ 1 (+ 2 (+ 3 (my-sum empty))))
= (+ 1 (+ 2 (+ 3 0)))
= (+ 1 (+ 2 3)) = (+ 1 5)
= 6
```

Sum of a List (Naïve Recursive)

```
1 (define (my-sum lst)
2   (cond
3     [(empty? lst) 0]
4     [else (+ (first lst)
5              (my-sum (rest lst)))]))
```

- Two branches: one for the empty list (base case), one for a non-empty list (recursion).
- Simple definition, easy to verify (e.g. via the substitution model).
- The recursive call is used as an argument to `+` (so `my-sum` is not tail recursive).
- This leads to a problem: a large expression is accumulated in memory before any addition can be computed.

Sum of a List (Tail Recursive, with an Accumulator Parameter)



```
1 (define (my-sum lst)
2   (define (helper lst current)
3     (cond
4       [(empty? lst) current]
5       [else (helper (rest lst)
6                     (+ current (first lst)))]))
7   (helper lst 0))
```

Idea (the **accumulator parameter** pattern, see Abelson and Sussman 1996, §1.2.1):

- Introduce a **tail recursive** helper function with an additional argument `current` that will represent local state (the intermediate sum).
- Update `current` when doing a recursive call.
- Call the helper from the main function with some initial state.

Sum of a List (Tail Recursive, with an Accumulator Parameter)

```
1  (define (my-sum lst)
2    (define (helper lst current)
3      (cond
4        [(empty? lst) current]
5        [else (helper (rest lst)
6                      (+ current (first lst)))]))
7    (helper lst 0))
```

```
(my-sum (list 1 2 3)) = (helper (list 1 2 3) 0)
```

Sum of a List (Tail Recursive, with an Accumulator Parameter)

```
1  (define (my-sum lst)
2    (define (helper lst current)
3      (cond
4        [(empty? lst) current]
5        [else (helper (rest lst)
6                      (+ current (first lst)))]))
7    (helper lst 0))
```

```
(my-sum (list 1 2 3)) = (helper (list 1 2 3) 0)
= (helper (list 2 3) (+ 0 1)) = (helper (list 2 3) 1)
```

Sum of a List (Tail Recursive, with an Accumulator Parameter)



```
1  (define (my-sum lst)
2    (define (helper lst current)
3      (cond
4        [(empty? lst) current]
5        [else (helper (rest lst)
6                      (+ current (first lst)))]))
7    (helper lst 0))
```

```
(my-sum (list 1 2 3)) = (helper (list 1 2 3) 0)
= (helper (list 2 3) (+ 0 1)) = (helper (list 2 3) 1)
= (helper (list 3) (+ 1 2)) = (helper (list 3) 3)
```

Sum of a List (Tail Recursive, with an Accumulator Parameter)



```
1  (define (my-sum lst)
2    (define (helper lst current)
3      (cond
4        [(empty? lst) current]
5        [else (helper (rest lst)
6                      (+ current (first lst)))]))
7    (helper lst 0))
```

```
(my-sum (list 1 2 3)) = (helper (list 1 2 3) 0)
= (helper (list 2 3) (+ 0 1)) = (helper (list 2 3) 1)
= (helper (list 3) (+ 1 2)) = (helper (list 3) 3)
= (helper (list) (+ 3 3)) = (helper (list) 6)
```

Sum of a List (Tail Recursive, with an Accumulator Parameter)



```
1  (define (my-sum lst)
2    (define (helper lst current)
3      (cond
4        [(empty? lst) current]
5        [else (helper (rest lst)
6                      (+ current (first lst)))]))
7    (helper lst 0))
```

```
(my-sum (list 1 2 3)) = (helper (list 1 2 3) 0)
= (helper (list 2 3) (+ 0 1)) = (helper (list 2 3) 1)
= (helper (list 3) (+ 1 2)) = (helper (list 3) 3)
= (helper (list) (+ 3 3)) = (helper (list) 6)
= 6
```

Accumulator Parameter Pattern vs Imperative Loop

Compare our Racket version with a Python **while**-loop:

```racket
; Racket (accumulator parameter)
(define (my-sum lst)
  (define (helper lst current)
    (cond
      [(empty? lst)
       current]
      [else
       (helper (rest lst)
               (+ current
                  (first lst)))]))
  (helper lst 0))
```

```python
# Python (while-loop)
def my_sum(lst):
  current = 0
  while True:
    if len(lst) == 0:
      return current
    else:
      current = current + lst[0]
      lst = lst[1:]
```

Sum of a List (via Higher-Order Functions)

```
1 (define (my-sum lst)
2   (apply + lst))

(my-sum (list 1 2 3 4))
```

Sum of a List (via Higher-Order Functions)

```
1 (define (my-sum lst)
2   (apply + lst))
```

```
(my-sum (list 1 2 3 4))
= (apply + (list 1 2 3 4))
```

Sum of a List (via Higher-Order Functions)

```
1 (define (my-sum lst)
2   (apply + lst))
```

```
(my-sum (list 1 2 3 4))
= (apply + (list 1 2 3 4))
= (+ 1 2 3 4)
```

Sum of a List (via Higher-Order Functions)

```
1 (define (my-sum lst)
2   (apply + lst))
```

```
(my-sum (list 1 2 3 4))
= (apply + (list 1 2 3 4))
= (+ 1 2 3 4)
= 10
```

Sum of a List (via Higher-Order Functions)

```
1 (define (my-sum lst)
2   (apply + lst))
```

```
(my-sum (list 1 2 3 4))
= (apply + (list 1 2 3 4))
= (+ 1 2 3 4)
= 10
```

More on higher-order functions next week.

The Model at Work: Is the Refactoring Safe?

Two versions of the same function, of different shape:

```
(define (my-sum lst)                                    ; naive
  (cond [(empty? lst) 0]
        [else (+ (first lst) (my-sum (rest lst)))]))
(define (helper lst current)                            ; accumulator
  (cond [(empty? lst) current]
        [else (helper (rest lst) (+ current (first lst)))]))
(define (my-sum/acc lst) (helper lst 0))
```

The Model at Work: Is the Refactoring Safe?

Two versions of the same function, of different shape:

```
(define (my-sum lst)                                    ; naive
  (cond [(empty? lst) 0]
        [else (+ (first lst) (my-sum (rest lst)))]))
(define (helper lst current)                            ; accumulator
  (cond [(empty? lst) current]
        [else (helper (rest lst) (+ current (first lst)))]))
(define (my-sum/acc lst) (helper lst 0))
```

Tests cover the lists you tried.

The model covers **every** list, if we trace with letters instead of numbers.

The Model at Work: Is the Refactoring Safe?

Two versions of the same function, of different shape:

```
(define (my-sum lst)                                  ; naive
  (cond [(empty? lst) 0]
        [else (+ (first lst) (my-sum (rest lst)))]))
(define (helper lst current)                          ; accumulator
  (cond [(empty? lst) current]
        [else (helper (rest lst) (+ current (first lst)))]))
(define (my-sum/acc lst) (helper lst 0))
```

Tests cover the lists you tried.
The model covers **every** list, if we trace with letters instead of numbers.

Claim

For every list `lst` and number `current`:
`(helper lst current) = (+ current (my-sum lst))`.

The Model at Work: A Proof for Every List

A list is `empty` or `(cons x rest)`, so there are two cases.

Case `empty`. Both sides are `current`:

The Model at Work: A Proof for Every List

A list is `empty` or `(cons x rest)`, so there are two cases.

Case `empty`. Both sides are `current`:

```
(helper empty current) = current
(+ current (my-sum empty)) = (+ current 0) = current
```

The Model at Work: A Proof for Every List

A list is `empty` or `(cons x rest)`, so there are two cases.

Case `empty`. Both sides are `current`:

```
(helper empty current) = current
(+ current (my-sum empty)) = (+ current 0) = current
```

Case `(cons x rest)`. Assume the claim for the shorter list `rest`.

The Model at Work: A Proof for Every List

A list is `empty` or `(cons x rest)`, so there are two cases.

Case `empty`. Both sides are `current`:

```
(helper empty current) = current
(+ current (my-sum empty)) = (+ current 0) = current
```

Case `(cons x rest)`. Assume the claim for the shorter list `rest`.

```
(helper (cons x rest) current)
= (helper rest (+ current x))
```

the **`else`** clause

The Model at Work: A Proof for Every List

A list is `empty` or `(cons x rest)`, so there are two cases.

Case `empty`. Both sides are `current`:

```
(helper empty current) = current
(+ current (my-sum empty)) = (+ current 0) = current
```

Case `(cons x rest)`. Assume the claim for the shorter list `rest`.

```
(helper (cons x rest) current)
= (helper rest (+ current x))                    the else clause
= (+ (+ current x) (my-sum rest))                the claim, for rest
```

The Model at Work: A Proof for Every List

A list is `empty` or `(cons x rest)`, so there are two cases.

Case `empty`. Both sides are `current`:

```
(helper empty current) = current
(+ current (my-sum empty)) = (+ current 0) = current
```

Case `(cons x rest)`. Assume the claim for the shorter list `rest`.

```
(helper (cons x rest) current)
= (helper rest (+ current x))                    the else clause
= (+ (+ current x) (my-sum rest))                the claim, for rest
= (+ current (+ x (my-sum rest)))                + is associative
```

The Model at Work: A Proof for Every List

A list is `empty` or `(cons x rest)`, so there are two cases.

Case `empty`. Both sides are `current`:

```
(helper empty current) = current
(+ current (my-sum empty)) = (+ current 0) = current
```

Case `(cons x rest)`. Assume the claim for the shorter list `rest`.

```
(helper (cons x rest) current)
= (helper rest (+ current x))                    the else clause
= (+ (+ current x) (my-sum rest))                the claim, for rest
= (+ current (+ x (my-sum rest)))                + is associative
= (+ current (my-sum (cons x rest)))             the else clause of my-sum, backwards
```

The Model at Work: A Proof for Every List

A list is `empty` or `(cons x rest)`, so there are two cases.

Case `empty`. Both sides are `current`:

```
(helper empty current) = current
(+ current (my-sum empty)) = (+ current 0) = current
```

Case `(cons x rest)`. Assume the claim for the shorter list `rest`.

```
(helper (cons x rest) current)
= (helper rest (+ current x))                    the else clause
= (+ (+ current x) (my-sum rest))                the claim, for rest
= (+ current (+ x (my-sum rest)))                + is associative
= (+ current (my-sum (cons x rest)))             the else clause of my-sum, backwards
```

So `(my-sum/acc lst) = (helper lst 0) = (my-sum lst)`, for every `lst`.

Exercise: Reverse a List with an Accumulator

```
(define (my-reverse lst)
  (cond
    [(empty? lst) empty]
    [else (append (my-reverse (rest lst))
                  (list (first lst)))]))
```

Task

Rewrite it with a tail-recursive helper and an accumulator.
Halfway through `(list 1 2 3 4)`, what does the accumulator hold?

Exercise: Reverse a List with an Accumulator

```
(define (my-reverse lst)
  (cond
    [(empty? lst) empty]
    [else (append (my-reverse (rest lst))
                  (list (first lst)))]))
```

Task

Rewrite it with a tail-recursive helper and an accumulator.
Halfway through `(list 1 2 3 4)`, what does the accumulator hold?

Hint

Reversing is putting each element in front of the ones already seen: `cons`.

Exercise: Reverse a List with an Accumulator (Solution)

```
(define (my-reverse lst)
  (define (helper lst acc)
    (cond
      [(empty? lst) acc]
      [else (helper (rest lst) (cons (first lst) acc))]))
  (helper lst empty))
```

Exercise: Reverse a List with an Accumulator (Solution)

```
(define (my-reverse lst)
  (define (helper lst acc)
    (cond
      [(empty? lst) acc]
      [else (helper (rest lst) (cons (first lst) acc))]))
  (helper lst empty))
```

```
(my-reverse (list 1 2 3)) = (helper (list 1 2 3) '())
```

Exercise: Reverse a List with an Accumulator (Solution)

```
(define (my-reverse lst)
  (define (helper lst acc)
    (cond
      [(empty? lst) acc]
      [else (helper (rest lst) (cons (first lst) acc))]))
  (helper lst empty))
```

```
(my-reverse (list 1 2 3)) = (helper (list 1 2 3) '())
= (helper (list 2 3) '(1)) = (helper (list 3) '(2 1))
```

Exercise: Reverse a List with an Accumulator (Solution)

```
(define (my-reverse lst)
  (define (helper lst acc)
    (cond
      [(empty? lst) acc]
      [else (helper (rest lst) (cons (first lst) acc))]))
  (helper lst empty))
```

```
(my-reverse (list 1 2 3)) = (helper (list 1 2 3) '())
= (helper (list 2 3) '(1)) = (helper (list 3) '(2 1))
= (helper '() '(3 2 1)) = '(3 2 1)
```

Exercise: Reverse a List with an Accumulator (Solution)

```
(define (my-reverse lst)
  (define (helper lst acc)
    (cond
      [(empty? lst) acc]
      [else (helper (rest lst) (cons (first lst) acc))]))
  (helper lst empty))
```

```
(my-reverse (list 1 2 3)) = (helper (list 1 2 3) '())
= (helper (list 2 3) '(1)) = (helper (list 3) '(2 1))
= (helper '() '(3 2 1)) = '(3 2 1)
```

`append` copies its first argument: the naïve version does about $n^2/2$ steps on n elements, the helper n.

Quick recap

1. What is the **substitution model** for, and what is it not?
2. In which order does **Racket** evaluate a function call?
3. Where does a name in a function body get its **meaning**?
4. Why does **tail recursion** help?
5. What does an **accumulator parameter** carry?

Quick recap

1. What is the **substitution model** for, and what is it not?
   Reason about what a call evaluates to; not how the interpreter works.
2. In which order does **Racket** evaluate a function call?
3. Where does a name in a function body get its **meaning**?
4. Why does **tail recursion** help?
5. What does an **accumulator parameter** carry?

Quick recap

1. What is the **substitution model** for, and what is it not?
   Reason about what a call evaluates to; not how the interpreter works.
2. In which order does **Racket** evaluate a function call?
   Arguments first: call by value, or applicative order in SICP's words.
3. Where does a name in a function body get its **meaning**?

4. Why does **tail recursion** help?

5. What does an **accumulator parameter** carry?

Quick recap



1. What is the **substitution model** for, and what is it not?
   Reason about what a call evaluates to; not how the interpreter works.
2. In which order does **Racket** evaluate a function call?
   Arguments first: call by value, or applicative order in SICP's words.
3. Where does a name in a function body get its **meaning**?
   Where the function was written: lexical scope.
4. Why does **tail recursion** help?

5. What does an **accumulator parameter** carry?

Quick recap

1. What is the **substitution model** for, and what is it not?
   Reason about what a call evaluates to; not how the interpreter works.
2. In which order does **Racket** evaluate a function call?
   Arguments first: call by value, or applicative order in SICP's words.
3. Where does a name in a function body get its **meaning**?
   Where the function was written: lexical scope.
4. Why does **tail recursion** help?
   Nothing is left to do after the call, so no expression piles up.
5. What does an **accumulator parameter** carry?

Quick recap

1. What is the **substitution model** for, and what is it not?
   Reason about what a call evaluates to; not how the interpreter works.
2. In which order does **Racket** evaluate a function call?
   Arguments first: call by value, or applicative order in SICP's words.
3. Where does a name in a function body get its **meaning**?
   Where the function was written: lexical scope.
4. Why does **tail recursion** help?
   Nothing is left to do after the call, so no expression piles up.
5. What does an **accumulator parameter** carry?
   The local state, passed along instead of assigned.

Quiz for Lecture 2

- Ten minutes, on Moodle, now.
  Seven questions on this lecture, then three about the lecture itself.
- **No AI.** The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

Quiz for Lecture 2

- Ten minutes, on Moodle, now.
  Seven questions on this lecture, then three about the lecture itself.
- **No AI.** The quizzes are **7%** of the course between them, and the cheapest mark of the semester.

→ **The last three questions**

> Say what you learned and what stayed unclear, **specifically**.
> I read them before writing the next lecture.

What's Next

In today's lab you will

- write the worksheet on this lecture
- defend problem set 1, if your name is drawn
- start on problem set 2

→ **Before next week: higher-order functions**

- Read SICP §1.2 (Abelson and Sussman 1996) and solve exercises 1.11, 1.14, 1.16, and 1.26.
- Read *Racket Essentials 2.3*, at `https://docs.racket-lang.org/guide/`.
- Try writing a Racket function that renders the Koch snowflake.

Thank you!



References



Abelson, Harold and Gerald Jay Sussman (1996). ***Structure and interpretation of computer programs.*** The MIT Press.